



OpoGenius

Your seat, your goal

CASE STUDY

Building an iOS app for Spanish civil-service candidates
without knowing how to code

Jasper van Puffelen López

Founder and author of OpoGenius

June 2026

Contents

Contents	2
Executive summary	4
Context and motivation.....	5
1.1 The observed problem	5
1.2 Why this project	5
The product	6
2.1 The five tabs	6
Home · Dashboard.....	6
Calendar	7
Library.....	8
Statistics	9
Profile	10
2.2 Feature map	11
2.3 Deliberate design decisions.....	11
Stack and tools	12
3.1 Creation tools	12
3.2 Technical stack	12
3.3 Real financial investment	13
The build process	14
4.1 An honest timeline	14
July 2025 · Origin	14
August – November 2025 · Pause	14
Late 2025 – June 2026 · Restart and production.....	14
June 2026 · Release	14
4.2 How the work splits between human and AI	15
Risks of using AI and how they were managed	16
5.1 Unwarranted trust in generated code.....	16
5.2 Technical debt accumulated through inexperience	16
5.3 Loss of product coherence	16
5.4 Privacy and user data	16
5.5 Attribution and honesty	16
SWOT analysis	17
Lessons learned	18
7.1 On AI as a collaborator	18
7.2 On building a product as a non-technical person.....	18
7.3 On purpose.....	18

An honest roadmap..... 19

 8.1 Short term (coming months) 19

 8.2 Medium term (under exploration) 19

 8.3 No explicit commitment 19

 8.4 The open question..... 19

Closing 20

Executive summary

OpoGenius is a native iOS app for Spanish civil-service exam candidates. It organizes the syllabus by topic, applies adaptive spaced repetition based on the SM-2 algorithm, gamifies progress with XP, levels, streaks and daily missions, and offers detailed statistics and an effort calendar in heatmap format.

What makes the project singular is not the product itself, but how and by whom it was built:

- A person with no technical training and no prior programming experience.
- Relying entirely on artificial intelligence tools.
- As a side project, in personal time, while working full-time in another sector and another country.
- With a total investment of under 500 euros.
- No team, no investors, no monetization pressure.

This document is not a business plan. There is no funding round to justify, nor revenue target to defend. OpoGenius is a personal project, free, given to a specific community: close friends preparing for civil-service exams.

This case study documents three things: what the product is, how it was built, and what this process says about the new relationship between creativity, execution and the technical barrier in the age of AI.

The technical barrier that for decades separated creative people from functional digital products is dissolving. OpoGenius is a personal proof of it.

CHAPTER 1

Context and motivation

1.1 The observed problem

Spain is going through a particular situation in its labour market. Precariousness and uncertainty have led a growing number of young people to pursue public-sector employment as a route to stability. Enrolment figures for public competitive exams have risen steadily in recent years.

Preparing for a civil-service exam in Spain involves:

- Long processes: one to five years of preparation is the norm.
- Individual study, largely solitary.
- A massive volume of material that must stay fresh in memory for months or years.
- The Ebbinghaus curve operating relentlessly: what is not reviewed is forgotten within days.

The tools most candidates use are, in essence, the same as twenty years ago: printed paper, hand-drawn outlines, review by intuition. There are generic productivity apps and flashcard apps, but none is designed around the specific logic of the Spanish civil-service exam process: extensive syllabi, real exam dates, sustained review over years, and the psychological need to keep motivation up when the reward is months or years away.

1.2 Why this project

OpoGenius was not born as a market opportunity. It was born from a personal observation and a recently acquired capability.

The observation: several close friends are preparing for exams right now. I have seen the toll up close. I have seen how the lack of a system turns effort into anxiety, and how the absence of clear feedback on progress erodes consistency even in brilliant, disciplined people.

The capability: the arrival of language models good enough that a non-technical person can execute ideas that once belonged exclusively to the domain of developers. What was a wall for years is today a ramp.

The motivation is twofold. On one hand, to build a concrete tool that can serve concrete people. On the other, to prove to myself, and to leave public evidence, that the combination of curiosity, judgement and AI tools makes it possible to realize products that were until now out of reach.

This is a project I could not have done two years ago. The difference is not that I now know how to code — I still do not. The difference is that I can now direct the process.

CHAPTER 2

The product

OpoGenius is designed around one principle: the candidate needs a system, not more content. The material is supplied by the user. The intelligence of the system lies in managing when, how and with what intensity that material should be revisited.

2.1 The five tabs

The app is organized into five main views accessible from the bottom bar:

Note • The screenshots accompanying this section correspond to a beta version under development. They do not represent the final visual design: colours, text, sample data and layout may change in the released version.

Home · Dashboard

The landing view. It shows the personalized greeting, the user avatar (an evolving Canarian lizard), the current streak, the all-time record, the day's urgent review mission with its XP reward, and progress toward the next level.



Home view (beta): mission of the day, active streak and level progress bar.

Calendar

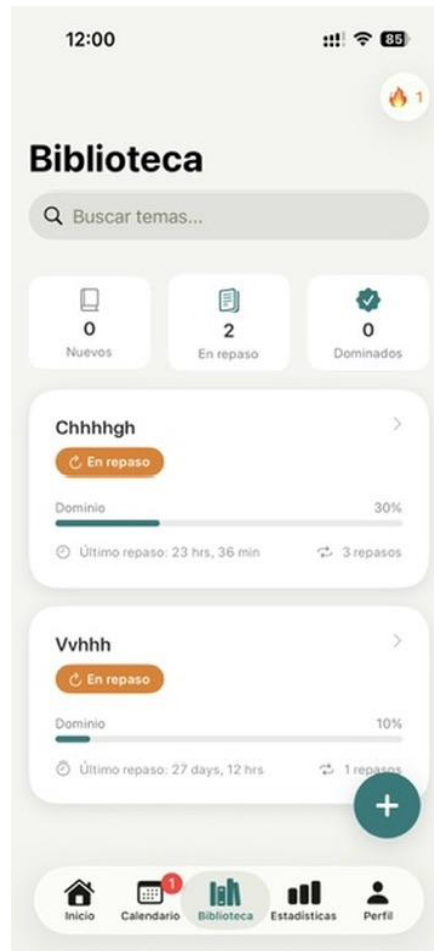
It lists pending reviews according to the spacing algorithm and shows a monthly effort map in heatmap style, inspired by the GitHub activity graph. Consistency becomes visible. A blank week shows. So does a solid streak.



Calendar view (beta): pending reviews and activity heatmap.

Library

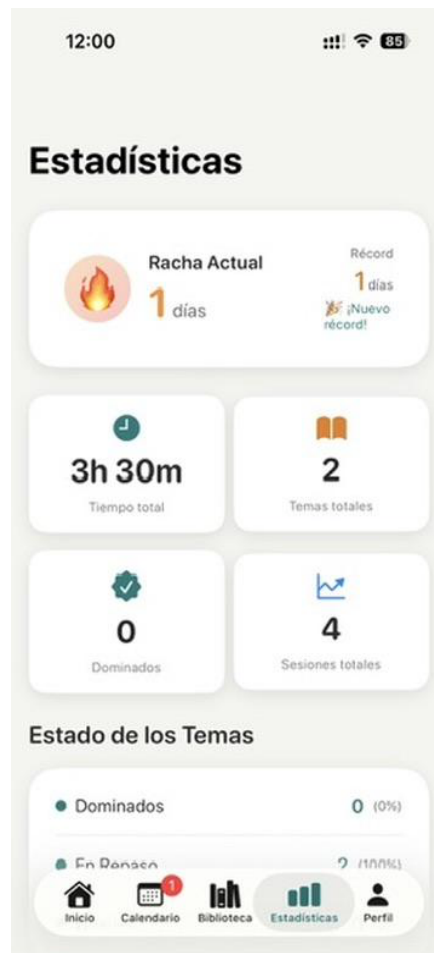
The user creates their own topics. The app does not impose a fixed catalogue: it adapts to any exam because the content is supplied by the candidate. Each topic shows its mastery percentage, the time since the last review and the number of accumulated reviews.



Library view (beta): topics categorized as New, In review or Mastered.

Statistics

Total study time, number of topics, mastered topics, completed sessions, current streak and record. The goal is not to create metric anxiety, but to give honest feedback on where things stand.



Statistics view (beta): a dashboard on the evolution of study.

Profile

Evolving avatar, current level, accumulated XP and historical statistics. The avatar — a Canarian lizard — changes form with progress. It is an identity nod from the author (a Canary Islander) rather than a market decision: it adds character without alienating other users.



Profile view (beta): evolving avatar and total statistics.

2.2 Feature map

Each feature has a specific purpose, tied to a real problem faced by the candidate:

Feature	What it does	Problem it solves
SM-2 spaced repetition	Calculates the optimal moment to review each topic	Progressive forgetting (Ebbinghaus)
Personal library	The user adds and organizes their topics	Lack of adaptability in generic apps
Heatmap calendar	Visual map of daily consistency	Invisibility of sustained effort
Daily missions	A concrete task of the day with an XP reward	Friction of deciding what to study
Levels and XP system	Measurable, gamified progress	Loss of motivation over the long term
Streaks	Counter of consecutive days	Consistency as a habit
Per-topic statistics	Mastery, time and sessions	Studying without knowing what is mastered
Evolving avatar	A mascot that changes with progress	Need for a personal symbol of progress

2.3 Deliberate design decisions

- **iOS only, native only.** Built with Swift 6 and SwiftUI. No cross-platform, no web, no PWA. The quality of the experience matters more than reach. A worse app in more places is a worse strategy than an excellent app in one place.
- **No backend, no accounts.** Local persistence in JSON and UserDefaults. The app works offline and user data never leaves the device. Zero servers to maintain, zero complex privacy policy, zero recurring infrastructure cost.
- **MVVM architecture.** A modern standard pattern in SwiftUI. Maintainable, testable, aligned with the ecosystem's conventions.
- **Visual identity with character.** A cream, teal and orange-accent palette. Typography mixing a display serif (headings) with a body sans-serif. The app does not look like generic productivity apps; it has a face of its own.
- **Free, no ads, no in-app purchases.** No monetization is planned. The app is given away. This is not a strategic growth decision; it is a personal decision about the purpose of the project.

CHAPTER 3

Stack and tools

The set of tools used during development says more about how OpoGenius was built than any other section of this document. Each tool covers a specific function within a non-traditional workflow.

3.1 Creation tools

Tool	Role in the project
Google AI Studio	Fast visual prototyping of the first beta. It allowed iterating on the idea before committing to native development.
Gemini + Nano Banana	Generation of brand imagery: icon, wordmark, avatar illustrations, visual assets.
Claude	Design of the app structure and tab architecture. Main reasoning assistant for product and code decisions.
Xcode	Apple’s native development environment. It is where the generated and reviewed code is compiled, tested in the simulator and prepared for the App Store.
Wispr Flow	Voice dictation to write prompts fluidly. It drastically reduces the friction of communicating complex intentions to language models.
Swift 6 + SwiftUI	The app’s language and framework. The modern languages of iOS.

3.2 Technical stack

Layer	Technology
Language	Swift 6
UI	SwiftUI
Architecture	MVVM
Concurrency	Async / Await
Persistence	JSON + UserDefaults (local)
Platform	Native iOS (iPhone)
Backend	None

3.3 Real financial investment

Item	Approximate amount
Apple Developer Program (annual)	€100
AI subscriptions (annual)	€200 – €300
Development machine (MacBook Pro)	Pre-existing personal investment
Marketing	€0
Cloud infrastructure	€0
External human team	€0
Direct project total	~ €400

The figure is deliberately small. It is not an efficiency metric to claim, but the natural consequence of a project with no team, no marketing and no recurring infrastructure. What is relevant: three years ago, a functional iOS app with this product surface would have required hiring an iOS developer for several thousand euros, or investing hundreds of hours learning to code. Neither of those two conditions was met here.

CHAPTER 4

The build process

This section is probably the most relevant in the document. Not to show off the project, but because its real timeline — with its pauses, hardware changes and intermediate decisions — is what makes this case replicable or learnable.

4.1 An honest timeline

July 2025 · Origin

The project starts after observing the pressure of civil-service exams in my close circle and, in parallel, a growing interest in generative AI tools. The problem is identified, validated as worthwhile, and a first exploratory visual version is built in Google AI Studio. It is not a real app, but a navigable prototype that serves to validate the idea without committing resources.

August – November 2025 · Pause

An international move. A change of country, a change of job, a change of everyday language. The project is frozen. Not for lack of interest, but for lack of available mental capacity. Accepting that a project lies fallow without abandoning it is, in hindsight, one of the most relevant decisions of the process. Forcing it at that moment would have produced worse work and burnout.

Late 2025 – June 2026 · Restart and production

Two changes coincide: replacing the MacBook Air M1 with a MacBook Pro removes the hardware friction, and the AI models available by subscription are significantly better than those of six months earlier. The project resumes with a clear scope and the real app is built in Swift / SwiftUI, assisted by Claude for structural design and by the other tools for specific tasks.

June 2026 · Version 1.0

Final version ready. Pending release on the App Store.

4.2 How the work splits between human and AI

A reasonable and honest question is: what exactly does the person do in a project like this? The answer, unvarnished:

Area	Who decides / who executes
Idea, problem, purpose of the project	Human. The AI does not decide what to build.
Brand identity, tone, palette, naming	Human directs, AI generates variants.
Overall architecture and tab structure	Human defines intent, AI proposes, human reviews.
Visual design of each screen	Human defines hierarchy and style, AI implements.
Swift / SwiftUI code line by line	AI writes most of it, human reviews, tests and iterates.
Bug resolution	Iterative between the two. The human spots the symptom, the AI proposes cause and fix, the human validates.
Spaced-repetition algorithm	Human chooses the model (SM-2), AI translates it into code.
Product decisions in grey areas	Human. Here the AI is a poor consultant if the decision is delegated to it.
Visual assets (icon, illustrations)	Generative AI with human direction.
Testing in simulator and on a real device	Human.

The human's work in this flow is not to write code, it is to exercise judgement: what to build, in what order, when to accept the AI's proposal and when to reject it, what to polish and what to let pass. The AI is an incredibly productive collaborator but without its own context of the project or any direction. Without someone steering, the AI produces mediocre code with high confidence. That is the main trap.

CHAPTER 5

Risks of using AI and how they were managed

Building a product relying mainly on generative AI opens up new ways to fail. Ignoring them would be naive. These are the real risks I identified in the process and how I dealt with them.

5.1 Unwarranted trust in generated code

The AI produces code that looks correct and compiles. It is not always correct. Models make subtle errors: inverted logic, race conditions, incorrect state handling, calls to deprecated APIs. Mitigation: manually test every flow before accepting it. Compiling is not validating.

5.2 Technical debt accumulated through inexperience

Without technical training, it is easy to accept suboptimal architectures because they "work". Mitigation: ask the AI to explain the why of each relevant architectural decision, and compare it with documented conventions of the iOS ecosystem. Future maintainability as an explicit criterion, not an implicit one.

5.3 Loss of product coherence

Each conversation with the AI starts practically from scratch. Without a human holding the thread, Monday's decisions do not inform Tuesday's. Mitigation: document product decisions in a project file the AI can refer to, and keep a clear, written brand vision.

5.4 Privacy and user data

An app can leak data through carelessness in the code. Design decision: 100% local persistence, no backend, no telemetry. This drastically reduces the risk surface. What never leaves the device cannot be leaked.

5.5 Attribution and honesty

It is tempting to present a project like this as "made by me" without nuance. It is more honest, and in the long run more useful professionally, to be clear about the role of AI. This document is an example of that transparency. The interesting question is not whether I coded it myself or not — I did not. It is whether I directed the process with judgement. That one, yes.

CHAPTER 6

SWOT analysis

A light strategic analysis, adjusted to the reality of the project: it does not aim to position a company against competitors, but to honestly map the terrain in which it is released.

WEAKNESSES	THREATS
• A one-person team with no formal technical training. • Maintenance capacity limited by part-time dedication. • No marketing budget. • No prior validation metrics (no beta user base). • iPhone only: excludes Android users.	• Changes in App Store policies. • Emergence of better-funded competitors. • AI models becoming more expensive or restricting usage. • Saturation of the educational-app market. • Personal risk of losing motivation, since it is an unpaid project.
STRENGTHS	OPPORTUNITIES
• Specific focus on the Spanish exam candidate, not a generic app. • Polished UX with a distinctive visual identity. • No recurring infrastructure cost. • Local persistence: privacy as a competitive advantage. • Scientifically proven review algorithm (SM-2). • Built by someone who understands the problem from outside the cliché.	• Growing demand for civil-service exams in Spain. • A real gap in digital tools adapted to the candidate. • AI enables far faster iteration than traditional competitors. • The project can serve as a professional showcase for the author. • A very active exam-candidate community on social media: fertile ground for future organic growth if pursued.

The honest reading of the SWOT: the weaknesses are real and acknowledged, with no attempt to disguise them. The opportunities exist but require an active decision to devote resources that are not committed today. As a personal project in its current form, OpoGenius operates within its strengths and accepts the external threats as conditions of the environment.

CHAPTER 7

Lessons learned

The most valuable thing about this project, beyond the product, are the conclusions I draw from the process. Some are technical, others are about how to work with AI, and others are personal.

7.1 On AI as a collaborator

- **The bottleneck has changed.** It is no longer knowing how to write code. It is knowing what to ask for and recognizing when what comes back is good.
- **Judgement becomes more important, not less.** When producing is cheap, what becomes scarce is deciding what is worth producing.
- **AI is a better executor than strategist.** Ask it for a product decision and it will propose something. That proposal is average. Good product decisions remain human.
- **Iterating by voice speeds up everything.** Wispr Flow has changed my relationship with the models. Speaking is faster than typing and produces prompts richer in context.

7.2 On building a product as a non-technical person

- **The "hello world" is no longer the barrier.** The barrier is sustaining a project over months without it collapsing under its own complexity.
- **Accepting the pause was key.** The fallow months during the move were not wasted time. They were time spent not breaking something that was not yet ready to withstand an international move.
- **Hardware matters more than it seems.** Switching from an Air M1 to a Pro was not a whim: the friction of compiling and iterating on a limited machine erodes willpower far more than an occasional spike does.

7.3 On purpose

One of the things that has surprised me most is how much the quality of the work changes when the project has no commercial pressure. With no rush to monetize, no metrics to defend, no investors to satisfy, the only yardstick is: does this serve what it should serve? Would I proudly give it to a friend? Those questions, which seem simple, are a compass that is hard to hold on to when money is involved.

OpoGenius is not a business. It is a statement. Public, voluntary and, I hope, useful.

CHAPTER 8

An honest roadmap

What follows are not promises to investors or users. It is what I currently see as the possible evolution of the project, with its real level of commitment.

8.1 Short term (coming months)

- Actual release on the App Store.
- Direct distribution to the close circle of friends preparing for exams.
- Collecting qualitative feedback and fixing bugs.
- Small UX improvements based on what the close community points out.

8.2 Medium term (under exploration)

- Test mode with exam-style questions and timed mock exams.
- Improvements to the review algorithm, adjusted to real exam dates.
- Community features: shared streaks, comparison between candidates.
- A possible Android version, conditional on time and real demand.

8.3 No explicit commitment

- Monetization. Not ruled out, but not prioritized. If it is ever introduced, it would take the form of an optional premium plan, keeping the core free.
- Team. The project remains solo-founder for now.
- Internationalization. OpoGenius is designed for the Spanish civil-service exam system; a leap to other markets would mean redesigning the model of the problem.

8.4 The open question

The most interesting question for me is not "how many users will OpoGenius have in a year?". It is "what project will I build next, now that I know I can build this?". OpoGenius is the first piece of a possible side career as a digital-product creator. It is a bridge more than a destination.

Closing

If you have made it this far: thank you. OpoGenius is, in equal measure, an app for Spanish exam candidates and a personal experiment about what a curious, non-technical person, well supported by AI, can build today.

The app will be available soon on the App Store, free of charge. If you know someone preparing for civil-service exams in Spain, this project is, in part, also for them.

And if you yourself are wondering whether your idea is viable without knowing how to code: it is. What you need is not a programming course, it is to decide to start and to accept that the project will live with you for months, with its pauses, its hardware changes and its doubts.

I will end on the most personal note. For me, artificial intelligence has changed the rules. It excites me to be able to build things that one or two years ago were out of my reach, and it motivates me to think about what comes next.

I am convinced of one thing: when the technical barrier drops, curious and creative people do not stay the same — they become more capable. Technique stops being the filter. What counts becomes curiosity, judgement and the will to make things.

OpoGenius is the first project I build from that conviction. It will not be the last.



Jasper van Puffelen López

Founder and author of OpoGenius

OpoGenius — Your seat, your goal

June 2026